Storage & Cloud Infrastructure

Where your data lives

Last session

You packaged your app and its environment in an image, and ran a database next to it with Compose.

Two things we said on purpose:

- **Your data does not belong in the image.**
- The database kept its data in a **volume**... on your laptop.

So where should the project's data live? In which format? And how do you get to it from a container, a colleague's laptop or a server?

The sessions, updated

We lost 05/10, new schedule:

Date	Session
21/09/2026	GitHub: Tasks, Projects, Branches, VS Code
28/09/2026	Docker
19/10/2026	**Storage & Cloud infrastructure (today)**
21/10/2026	MLOps and CI/CD with GitHub Actions
26/10/2026	Experiment tracking and models with MLflow

Today

Part 1: what the cloud is, and where your data lives in it

Part 2: file formats: CSV, JSON, Parquet. Same data, very different results

Part 3: object storage, hands-on. S3 on your laptop, the same code for local and remote data, and the app from session 2 reading from it

Part 1

The cloud, and where data lives

Before the cloud





Companies **bought** servers, put them in a server room, and hired people to keep them alive. This is called **on-premise**.

- One big machine: a single point of failure
- A rack of them: does not grow faster than you can buy hardware

Problems with on-premise

- **Upfront cost**: you pay for hardware before you know what you need
- **Over-provisioning**: buy for the peak, pay for idle the rest of the year
- **Under-provisioning**: the peak arrives and there is no capacity
- **Lead time**: a new server takes weeks to arrive
- **Maintenance**: disks fail, patches, cooling, power, night shifts

For data science: a GPU server that sits idle between experiments, and storage that is always 90% full.

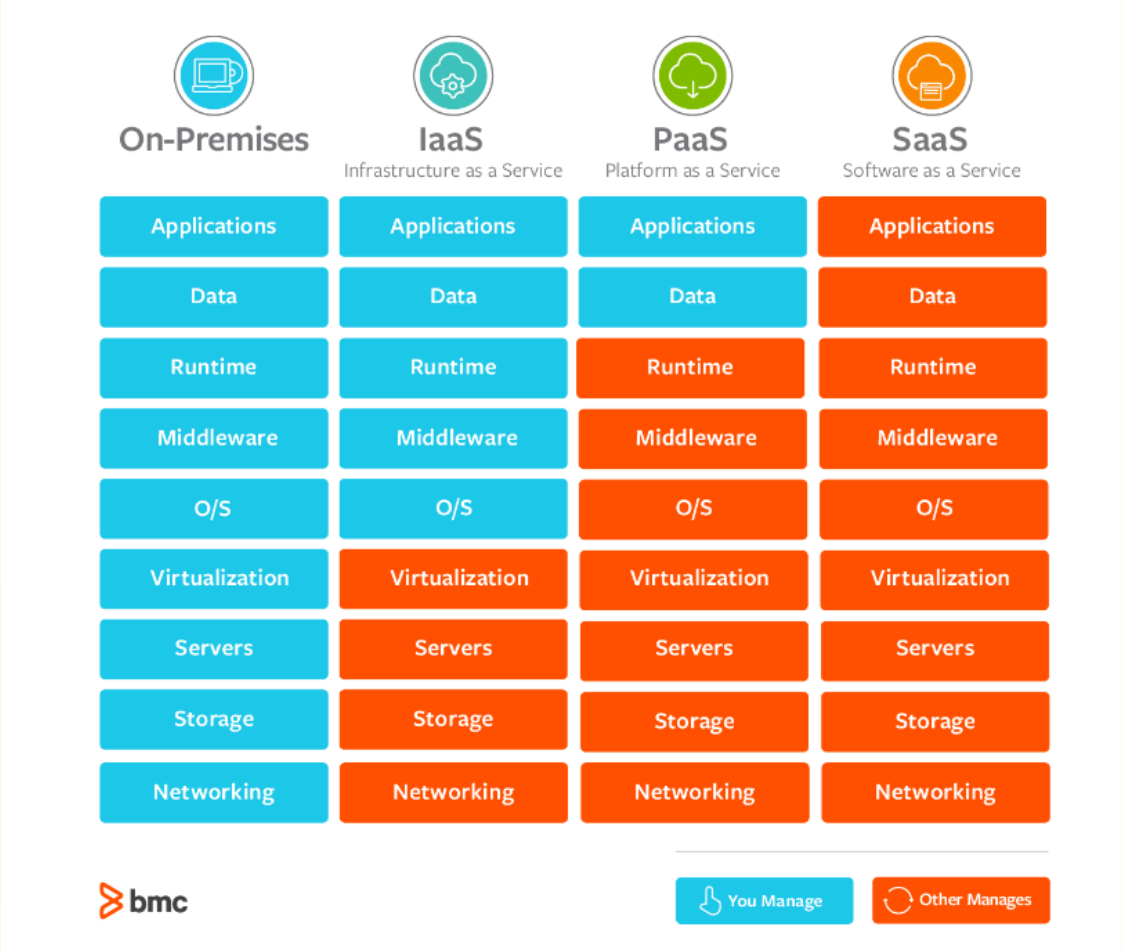
What is cloud computing?

Renting computing resources over the internet, instead of owning them.

- **On demand**: a machine in a minute, not a month
- **Elastic**: scale up for the experiment, scale down after
- **Pay per use**: per second of compute, per GB stored
- **Someone else's problem**: hardware, power, cooling, failed disks

AWS started in 2006. Azure and Google Cloud followed a few years later.

How much do you want to manage?



IaaS, PaaS, SaaS

	You get	You manage	Example
IaaS	Virtual machines, disks, networks	OS, runtime, your app	AWS EC2, a GPU VM
PaaS	A place to run your code or container	Your app	Google Cloud Run, Heroku
SaaS	A finished product	Nothing, you just use it	Gmail, GitHub, Colab

Object storage sits between IaaS and PaaS: you never see a server, you just use buckets.

The big three: same building blocks

	AWS	Google Cloud	Azure
Object storage	S3	Cloud Storage	Blob Storage
Virtual machines	EC2	Compute Engine	Virtual Machines
Run a container	ECS / Fargate	Cloud Run	Container Apps
Managed PostgreSQL	RDS	Cloud SQL	Database for PostgreSQL
Data warehouse	Redshift	BigQuery	Synapse / Fabric
ML platform	SageMaker	Vertex AI	Azure ML

Hundreds of services each, different names, **the same ideas**. Learn the idea once.

Compute: renting a machine

A virtual machine (EC2, Compute Engine, Azure VM) is a slice of a server in a data centre. You pick the size, it is ready in a minute.

Instance (AWS)	Size	Roughly, on demand
`t3.micro`	2 vCPU, 1 GB RAM	~0.01 USD/hour
`m7i.xlarge`	4 vCPU, 16 GB RAM	~0.20 USD/hour
`g5.xlarge`	4 vCPU, 16 GB RAM, 1 GPU	~1 USD/hour
8-GPU training machines	hundreds of GB RAM	tens of USD/hour

You pay per second **while it is on**, whether it is doing anything or not.

Your project's dataset

Renting a machine takes a minute. But where is your data right now?

- On your laptop, in `~/Downloads/data_final_v2.csv`
- In a Google Drive folder someone shared
- Committed to Git (please don't)
- On a server somebody set up and nobody remembers how

None of these answer: **can my teammate, my container and a server all read the same data, the same way?**

Three places data can live

	Your disk	A shared folder	Object storage
You reach it with	A file path	A file path, mounted over the network (NFS, SMB)	An HTTP request + credentials
Who can read it	You	Machines on that network	Anyone you give a key to
Who runs it	You	Your IT department	A cloud provider (or you, in Docker, today)
Grows to	Your laptop	A server's worth	Practically unlimited

Object storage is what the cloud runs on, but it is defined by **how you talk to it**, not by where it is. That is why we can run one on our laptops later.

Object storage

Store **objects** (bytes + metadata) under a **key**, inside a **bucket**. Talk to it over HTTP.

```
PUT    s3://taxi/raw/trips.parquet     ← upload
GET    s3://taxi/raw/trips.parquet     ← download
LIST   s3://taxi/raw/                  ← what is there?
DELETE s3://taxi/raw/trips.parquet
```

- Anyone with credentials and a network can read it
- Built for durability: designed for 99.999999999% (11 nines)
- You never think about disks, you pay per GB

Storage and compute are separate

- The machine's own disk disappears when you delete the machine. **That is fine**, nothing important lives there.
- The data lives in the **bucket**. Any number of machines, containers or laptops read the same copy.
- Each side scales on its own: a big GPU machine for one hour, then nothing, while the data costs cents per GB per month.

The machine is **disposable**. The data is not.

That is why the data is not in the image, and not on your laptop.

Regions and availability zones



A **region** is a geographic location (`eu-west-3` is Paris). Each region has several isolated **availability zones** (data centres).

Why the region matters to you

- **Latency**: data in Virginia, you in Barcelona: every read crosses the Atlantic
- **Law**: personal data of EU citizens has rules (GDPR) about where it can go
- **Cost**: prices differ per region, and moving data *between* regions is charged
- **Put compute next to data**: same region, or pay for it in time and money

When you create a bucket, you pick a region. Pick one on purpose.

What you pay for

	Roughly (AWS S3 Standard, check the pricing page)
Storage	~0.023 USD per GB per month
Requests	fractions of a cent per 1000 `PUT` / `GET` / `LIST`
Egress: data leaving the cloud	~0.09 USD per GB
Compute	per second the machine is **on**, used or not

A 1 TB dataset: ~23 USD/month to **store**. ~90 USD **each time** someone downloads all of it to their laptop.

Storage is cheap. **Moving data out and leaving things running** is what generates scary bills.

The classic mistakes

- A GPU machine left running over the weekend
- A bucket made **public** "just to test"
- Access keys committed to GitHub: bots scan public repos and use leaked keys within minutes
- Copying the whole dataset to every laptop, every day
- No budget alert, so you find out from the invoice

Set a **budget alert** the day you create an account. Every provider has one.

Credentials and access

The cloud needs to know **who you are** and **what you may do**: Identity and Access Management (**IAM**).

- **Who**: users for people, **roles** / service accounts for programs
- **What**: policies, e.g. "read-only on bucket `taxi`"
- **Least privilege**: the app that reads data cannot delete the bucket

Programs use an **access key ID** + **secret access key**: never in the image, never in Git, pass them as environment variables (session 2).

Do you need an account today? No. The **S3 API** is a de-facto standard, open source servers speak it, and we run one in Docker in Part 3. The extra material at the end walks through the same steps on real AWS.

Part 2

File formats

Our dataset

New York City yellow taxi trips, January 2024. Public, real, a decent size.

```
mkdir data
curl -L -o data/yellow_tripdata_2024-01.parquet \
  https://d37ci6vzurychx.cloudfront.net/trip-data/yellow_tripdata_2024-01.parquet
```

~50 MB. Pickup and dropoff times, distance, fare, tip, total, passengers… **2,964,624 trips, 19 columns.**

Set up Python

Today's practice lives in a repository: https://github.com/raulcd/ub-agile-data-science-storage. Create your own `ub-agile-data-science-storage` repo and follow its README.

```
python -m venv .venv
source .venv/bin/activate          # Windows: .venv\Scripts\activate
pip install -r requirements.txt    # pandas, pyarrow
```

- **pandas**: the DataFrame library you already know
- **pyarrow**: the Python library for Apache **Arrow**. It reads and writes Parquet, and talks to S3. We will see what Arrow is in a bit.

Load it

```
import pandas as pd

df = pd.read_parquet("data/yellow_tripdata_2024-01.parquet")
df.shape
```

```
(2964624, 19)
```

The file comes as Parquet. We will write it as CSV and JSON too, and compare.

CSV

```
VendorID,tpep_pickup_datetime,tpep_dropoff_datetime,passenger_count,...
2,2024-01-01 00:57:55,2024-01-01 01:17:43,1.0,1.72,1.0,N,186,79,2,...
1,2024-01-01 00:03:00,2024-01-01 00:09:36,1.0,1.8,1.0,N,140,236,1,...
```

Good: human-readable, every tool on earth opens it, Excel too.

Bad

- Everything is **text**: no types, `2024-01-01 00:57:55` is just a string
- No standard for nulls, quoting, separators, encodings
- To read one column you still read every row, all of it

JSON - One record per line

```
{"VendorID":2,"tpep_pickup_datetime":"2024-01-01T00:57:55.000",
 "passenger_count":1.0,"trip_distance":1.72,"store_and_fwd_flag":"N",...}
```

Good: nested data, readable, what every web API returns.

Bad: Repeats **every column name in every row**. Still text: dates are strings again. Slow to parse.

Great for APIs and logs. Bad for 3 million rows of a table.

Write the same data three ways

```
df.to_csv("data/trips.csv", index=False)
df.to_json("data/trips.json", orient="records", lines=True, date_format="iso")
df.to_parquet("data/trips.parquet")
```

```
ls -lh data/
```

What do you expect? Which is biggest? By how much?

Size

Format	Size	Write time
CSV	311 MB	10.3 s
JSON lines	1271 MB	9.7 s
Parquet	**61 MB**	**0.5 s**

Same 2,964,624 rows. JSON is **20x** bigger than Parquet, mostly column names repeated 3 million times.

On the cloud you pay for storage **and** for every byte you move. That factor is your bill.

Read time

```
import time

start = time.perf_counter()
pd.read_csv("data/trips.csv")
time.perf_counter() - start
```

Format	Read all	Read 2 columns
CSV	2.49 s	0.76 s
JSON lines	16.3 s	
Parquet	**0.07 s**	**0.03 s**

Measured on my laptop. Yours will differ, the ratios will not.

What came back

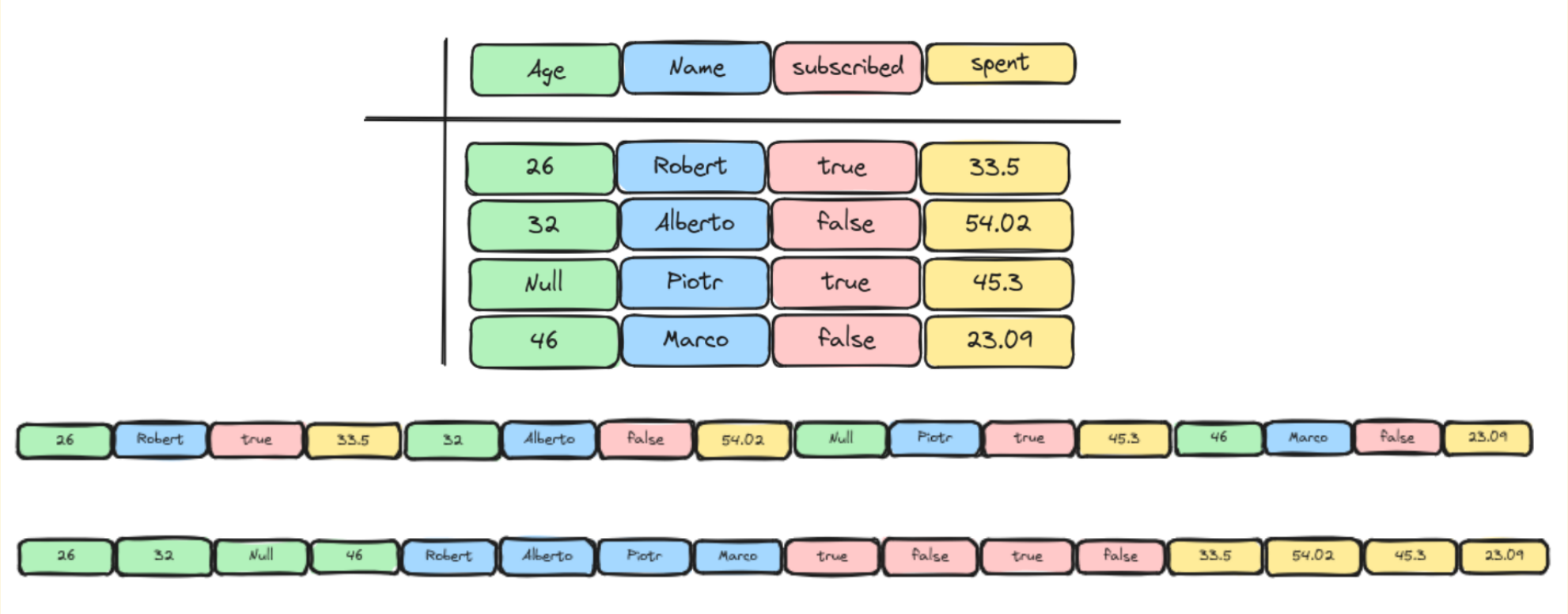
```
>> csv = pd.read_csv("data/trips.csv")
>> csv["tpep_pickup_datetime"].dtype
str
```

```
DtypeWarning: Columns (6: store_and_fwd_flag) have mixed types.
```

```
>> df["tpep_pickup_datetime"].dtype
datetime64[us]
```

CSV lost the types: dates are strings, and nulls turned a text column into a mix of `str` and `float`. Parquet stores the **schema**.

Why: rows vs columns



Row-oriented (CSV, JSON): one record after another.

Column-oriented (Parquet): all values of a column together.

Analytics reads columns

"Average fare per number of passengers":

```
df.groupby("passenger_count")["total_amount"].mean()
```

Needs **2 of 19 columns**.

- **CSV**: read every byte of every row, throw 17 columns away
- **Parquet**: jump to the 2 columns, read only those

```
pd.read_parquet("data/trips.parquet",
                columns=["passenger_count", "total_amount"])
```

Apache Parquet

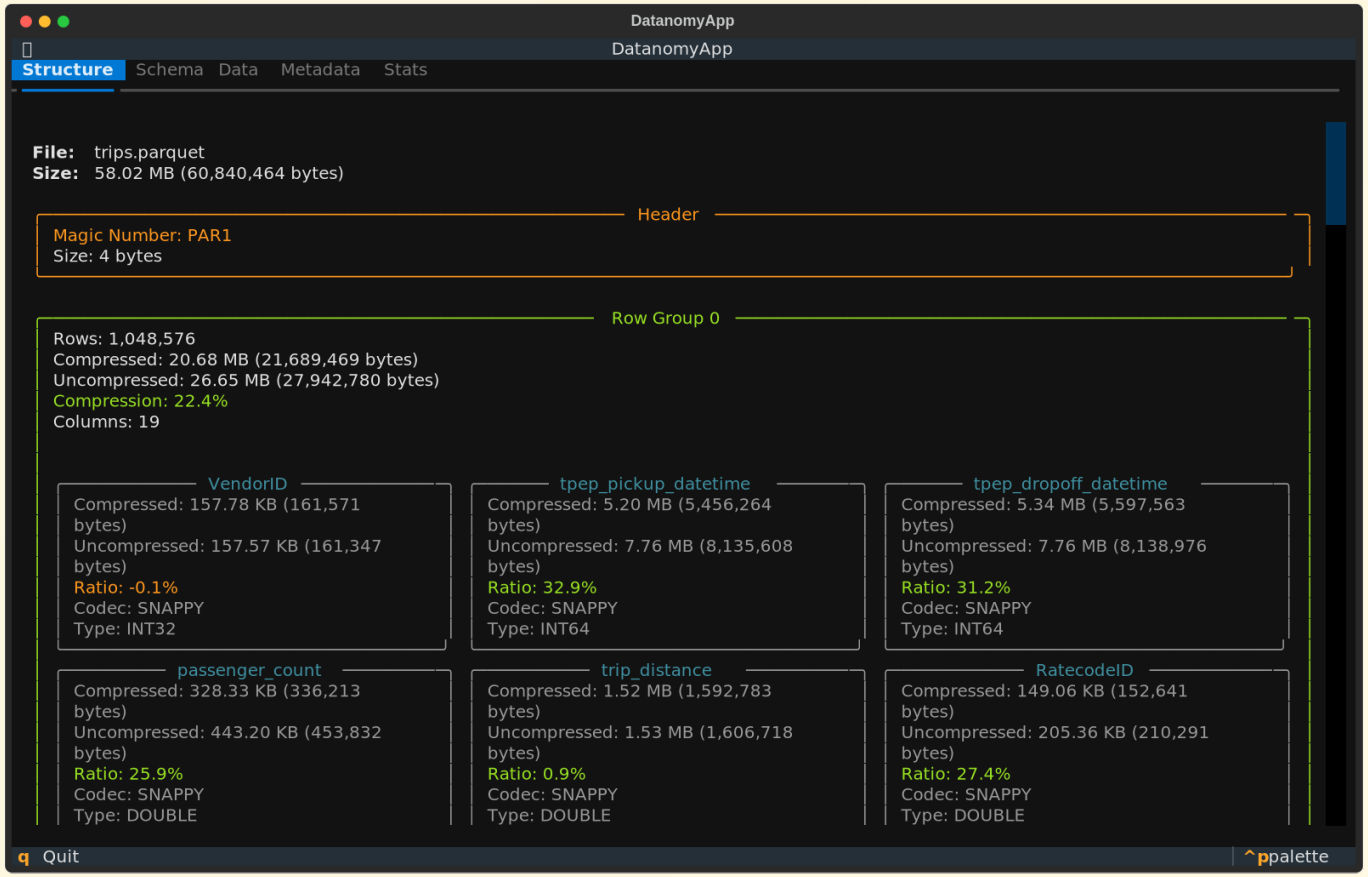
A **columnar file format** for analytics. Open source, created in 2013, now the default for data at rest.

- **Columnar**: read only the columns you need
- **Typed**: the schema travels with the file
- **Compressed**: per column (Snappy by default, Zstd is better)
- **Statistics**: min / max per chunk, so readers can skip data
- **Everywhere**: pandas, Polars, Spark, DuckDB, BigQuery, Snowflake…

Not human-readable. That is the trade-off.

Inside a Parquet file

```
uvx datanomy data/trips.parquet
```



Apache Arrow

Parquet is how columnar data is stored **on disk**.
Arrow is how columnar data is laid out **in memory**.

- One standard memory format, used by pandas, Polars, DuckDB, Spark…
- Tools can hand data to each other **without converting** it
- `pyarrow` is its Python library, and it is what pandas used under the hood today to read Parquet and, in a minute, S3

Disclaimer: this is what I work on. 🙂

Which format when?

Use	Format
Small table you want to look at or send by email	CSV
API responses, logs, nested documents	JSON
Anything analytical, anything big, anything you store	**Parquet**
Passing data between tools in memory	Arrow

For the project: **raw data as it arrives, everything you produce as Parquet.**

Exercise 1 - README - formats.py

1. Download the taxi dataset into `data/` (ignored by Git, why?)
2. Write it as CSV, JSON lines and Parquet. Compare sizes
3. Time reading each one, all columns and only 2
4. Check the dtypes of `tpep_pickup_datetime` after each read
5. Write the Parquet file again with `compression="zstd"`. Size?
6. Commit and push

Question: Someone sends you a 2 GB CSV every morning. What would you ask them to send instead, and why?

Part 3

Object storage, hands-on

Buckets and keys

A **bucket** is a top-level container. Its name is global (on AWS: in the whole world).

Inside, every object has a **key**: a string.

```
bucket: taxi
key:    raw/trips.parquet
URI:    s3://taxi/raw/trips.parquet
```

There are no directories

`raw/trips.parquet` is one string. The `/` is just a character.

Tools *show* folders by grouping keys that share a **prefix**. Delete the last object under `raw/` and the "folder" is gone.

Why should you care?

- "Rename a folder" means copy + delete **every object** in it
- Listing millions of keys is slow, and you pay per request
- Good layouts (prefixes) make data easy to find and skip

Objects are immutable

You `PUT` a whole object, you `GET` a whole object (or a byte range).

- No appending to a file
- No editing line 5 in place
- To change it: write a new version, replace the old one

That is why "a folder of Parquet files, each written once" is the natural shape of data on object storage.

Reads are **strongly consistent**: once a write succeeds, everyone sees it. (S3 only got this in December 2020.)

Local disk vs object storage

	Local disk	Object storage
Who can read it	You	Anyone with credentials
Latency	Microseconds	Tens of milliseconds per request
Throughput	One SSD	Scales with parallel requests
Size	Your disk	Practically unlimited
Durability	Until your laptop dies	11 nines, replicated
Cost	Already paid	Per GB-month + requests + egress
Edit in place	Yes	No, rewrite the object

Few big files are good. Millions of tiny files are slow and expensive.

Latency in human terms

If reading from the CPU cache took **1 second**...

Where the data is	Real latency	Human scale
CPU cache (L1)	~1 ns	**1 second**
RAM	~100 ns	**1.5 minutes**
SSD	~100 µs	**1 day**
Hard disk	~10 ms	**4 months**
Object storage (S3, GCS)	~10–100 ms per request	**4 months to 3 years**

Each request to a bucket costs as much as a disk seek, or more.

Hence:

few big files, only the columns you need, partitions you can skip.

S3 is the de-facto standard

Amazon S3 came first, and its **API** became the standard. Many systems speak it:

- Google Cloud Storage (interoperability mode), Cloudflare R2, Backblaze B2, OVH, ...
- Open source servers you run yourself: **RustFS**, SeaweedFS, Garage
- MinIO used to be the default for this, but stopped publishing free Docker images in 2025

The code you write against S3 works against any of them. Only the address (endpoint) and the credentials change.

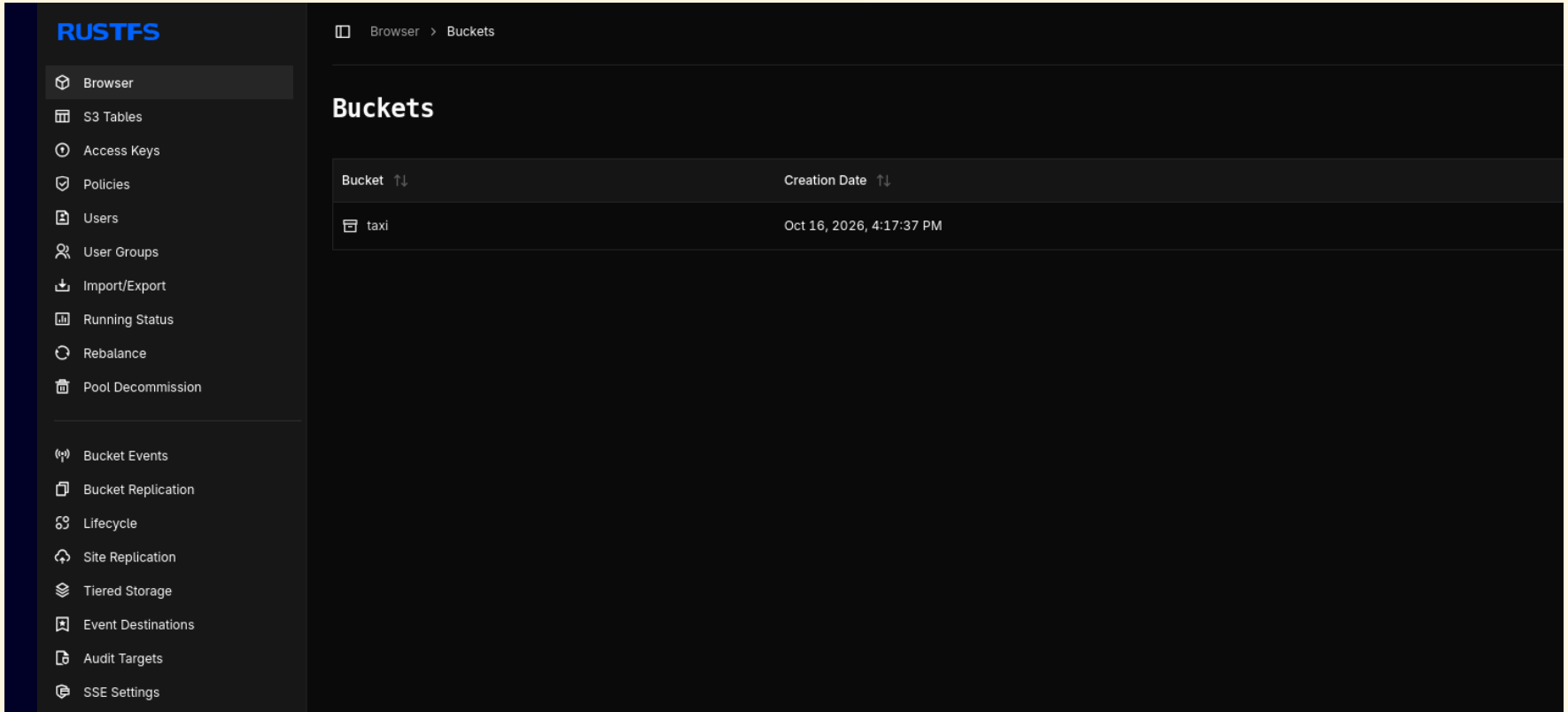
S3 on your laptop

We run **RustFS** in Docker. Open source, S3-compatible `docker compose up -d`.

```yaml
# compose.yaml
services:
  storage:
    image: rustfs/rustfs:1.0.1
    ports: ["9000:9000", "9001:9001"] # 9000 API; 9001 Web console
    environment:
      RUSTFS_ACCESS_KEY: student
      RUSTFS_SECRET_KEY: student-secret
    volumes: [s3data:/data]
volumes:
  s3data:
```

The web console

http://localhost:9001/rustfs/console/, log in with `student` / `student-secret`



Credentials as environment variables

The AWS libraries read credentials from standard variables. Set them in the terminal **before** you start `python`:

```
export AWS_ACCESS_KEY_ID=student
export AWS_SECRET_ACCESS_KEY=student-secret
export AWS_REGION=us-east-1
export S3_ENDPOINT=http://localhost:9000
```

`S3_ENDPOINT` is our own variable: where the S3 server is. On real AWS you would not set it.

Connect from Python

```
s3 = fs.S3FileSystem(
    endpoint_override=os.environ.get("S3_ENDPOINT"),
    allow_bucket_creation=True,
)
```

- `S3FileSystem()` picks up the keys and region from the environment
- `endpoint_override`: talk to our server, not to Amazon. `None` means real AWS
- `allow_bucket_creation`: off by default, so a typo in a path doesn't create buckets

A gotcha worth seeing once

Forget the endpoint, and pyarrow does exactly what you asked: it goes to **real AWS**, with your local credentials.

```
fs.S3FileSystem().get_file_info(fs.FileSelector("taxi"))
```

```
OSError: When listing objects under key '' in bucket 'taxi':
AWS Error INVALID_ACCESS_KEY_ID during ListObjectsV2 operation:
The AWS Access Key Id you provided does not exist in our records.
```

Somebody on Amazon owns a bucket called `taxi`. Not us.

If you see an error that mentions Amazon, check your endpoint first.

Upload

Create a bucket and copy the files from Part 2:

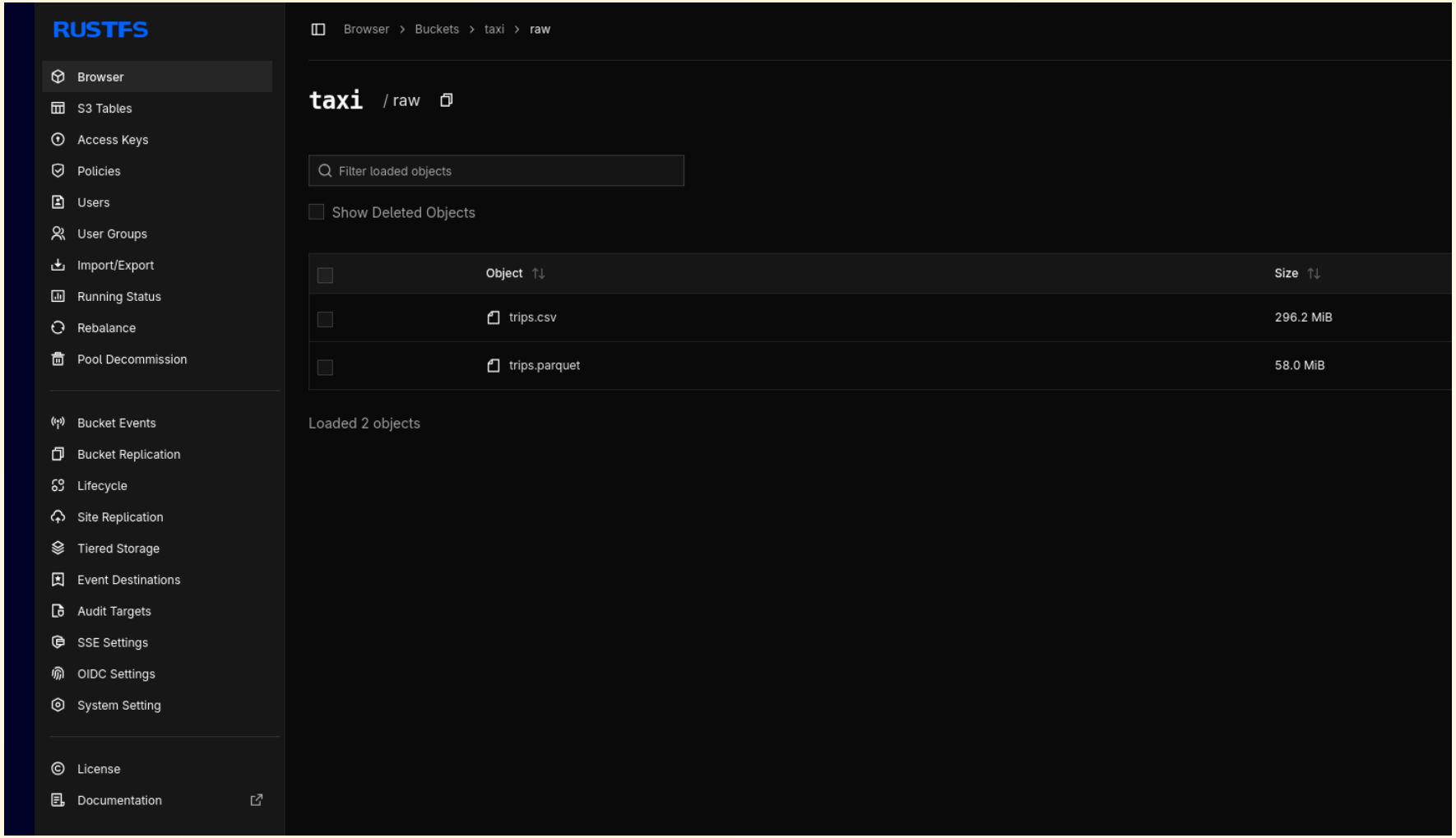
```
s3.create_dir("taxi")

fs.copy_files("data/trips.parquet", "taxi/raw/trips.parquet",
              destination_filesystem=s3)
fs.copy_files("data/trips.csv", "taxi/raw/trips.csv",
              destination_filesystem=s3)
```

Paths are `bucket/key`, with no `s3://` in front when you go through `s3`.

Refresh the web console: there they are.

Upload: in the console



List

```
for info in s3.get_file_info(fs.FileSelector("taxi", recursive=True)):
    print(info.path, info.type.name, info.size)
```

```
taxi/raw/trips.csv File 310608360
taxi/raw Directory None
taxi/raw/trips.parquet File 60840464
```

`taxi/raw` shows up as a directory. We never created it: pyarrow **infers** it from the shared prefix.

Read it back, same code

Local:

```
pd.read_parquet("data/trips.parquet")
```

Object storage:

```
remote = pd.read_parquet("taxi/raw/trips.parquet", filesystem=s3)
```

Read it back, same code

	Local	Object storage (local RustFS)
Parquet, all columns	0.07 s	0.13 s
Parquet, 2 columns	0.03 s	0.03 s
CSV	2.49 s	4.80 s

Only the **path** and the **filesystem** change. Your analysis code doesn't.

Partitioning

Most questions are about **a slice**: one day, one city, one client.

Split the data into one folder per value of a column:

```
taxi/by_day/pickup_day=2024-01-01/....parquet
taxi/by_day/pickup_day=2024-01-02/....parquet
...
taxi/by_day/pickup_day=2024-01-31/....parquet
```

Every tool understands **Hive-style** convention `column=value` in path.

A reader wants 15 January opens **one** folder and skips the others.

Partitioning: write

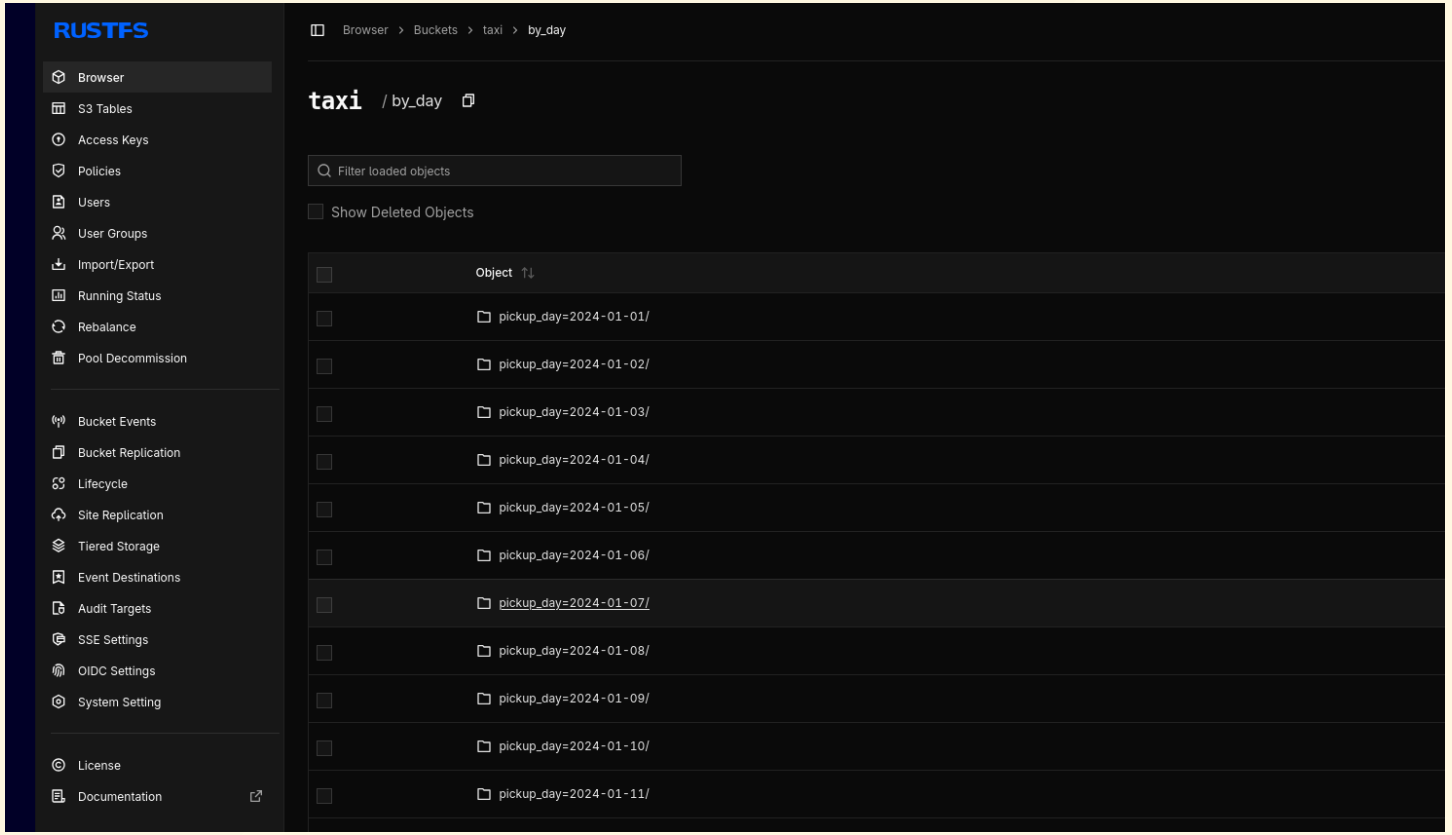
The dataset has a few trips with wrong dates (2002, 2009). Keep January only:

```
jan = df[df["tpep_pickup_datetime"].between("2024-01-01", "2024-02-01")].copy()
jan["pickup_day"] = jan["tpep_pickup_datetime"].dt.strftime("%Y-%m-%d")

jan.to_parquet("taxi/by_day", filesystem=s3, partition_cols=["pickup_day"],
               existing_data_behavior="delete_matching")
```

Without the last argument, running it twice **adds** a second file per day: twice the trips.

Partitioning: in the console



31 "folders", one per day. Remember: they are just key prefixes.

Partitioning: read

```
one_day = pd.read_parquet(
    "taxi/by_day",
    filesystem=s3,
    filters=[("pickup_day", "==", "2024-01-15")],
)
one_day.shape
```

```
(77033, 20)
```

0.05 s. One file opened out of 31.

Exercise 2

README, **Exercise 2**: `compose.yaml`, `upload.py`, `read_remote.py`

1. Start RustFS, open the web console
2. Set the environment variables
3. `upload.py`: bucket, raw files, partitioned dataset. Find in web
4. `read_remote.py`: read it back, time it, read one day with `filters=`
5. Commit and push

Questions: after `docker compose down`, is the data still there? And after `down -v`? What happens if you unset `S3_ENDPOINT`?

An app that reads the bucket

Same idea as the app from session 2, now answering questions about taxi trips. In the class repo:

```
ub-agile-data-science-storage/
├── compose.yaml           (one new service: web)
└── app/
    ├── Dockerfile         (as in session 2)
    ├── requirements.txt   (flask, pandas, pyarrow)
    └── app.py             (one route)
```

compose.yaml: the web service

Added next to `storage`:

```yaml
web:
  build: ./app
  ports:
    - "5001:5000"
  environment:
    S3_ENDPOINT: http://storage:9000
    AWS_ACCESS_KEY_ID: student
    AWS_SECRET_ACCESS_KEY: student-secret
    AWS_REGION: us-east-1
  depends_on:
    - storage
```

app.py

```
import pandas as pd
import pyarrow.fs as fs

s3 = fs.S3FileSystem(endpoint_override=os.environ.get("S3_ENDPOINT"))
BUCKET = os.environ.get("S3_BUCKET", "taxi")

@app.route('/trips/<day>')
def trips(day):
    df = pd.read_parquet(
        f"{BUCKET}/by_day",
        filesystem=s3,
        columns=["total_amount"],
        filters=[("pickup_day", "==", day)],
    )
    return f"{day}: {len(df)} trips, {df['total_amount'].mean():.2f} USD on average"
```

One partition, one column: ~170 KB of a 2 MB file, not the whole 61 MB.
The bucket name comes from the environment too. You will see why soon.

Run it

```
docker compose up -d --build
```

`storage` is the same service as in Exercise 2, with the same volume: the data you uploaded is still there.

Open http://localhost:5001/trips/2024-01-15:

```
2024-01-15: 77033 trips, 28.11 USD on average
```

Exercise 3

README, **Exercise 3**: `app/` and the `web` service

1. Create the three files in `app/`, add `web` to `compose.yaml`
2. `docker compose up -d --build`, open a few days in the browser
3. Which day of January had the most trips?
4. Change `S3_ENDPOINT` for `web` to `http://localhost:9000`. What happens, and why?
5. The credentials are in `compose.yaml`, in Git. OK here. And for real ones?
6. Commit and push

From your laptop to the cloud

What changes to run all of this on real AWS?

	Today	AWS
`S3_ENDPOINT`	`http://storage:9000`	not set
Credentials	`student` / `student-secret`	IAM keys, or a role
Region	anything	where the bucket lives
`S3_BUCKET`	`taxi`	a globally unique name
Your code		**unchanged**

What creating a real bucket and a machine looks like: in the extra material, *The real cloud, step by step*.

Try it yourself, no credit card

Azure for Students: sign up with your university email.

- 100 USD of credit for 12 months, no credit card
- Small VMs (B1s) free for 750 hours a month in the first year

AWS	Azure
S3 bucket	Storage account + **container**
EC2 instance	Virtual machine
IAM	Entra ID + role assignments

Same steps as the AWS walkthrough in the extra material: a container, a VM, `ssh`, `docker run`.

If you get stuck

- **Error mentions Amazon / InvalidAccessKeyId**: your endpoint is not set, you are talking to real AWS
- **Access denied**: wrong keys, or the variables were set in another terminal
- **Bucket not found**: create it, or `allow_bucket_creation=True`
- **Connection refused**: storage not running, or wrong host (`localhost` vs `storage`)
- `FileNotFoundError` **on read**: no `s3://` with `filesystem=s3`, and check the key in the console

If you want to learn more...

Extra material

The real cloud, step by step

On AWS: a bucket

AWS console → **S3** → **Create bucket**

- **Name**: globally unique, e.g. `ub-ads-taxi-2026`
- **Region**: `eu-south-2` (Spain). Close to us, data stays in the EU.
  It is an opt-in region: enable it once under Account → AWS Regions
- **Block all public access**: leave it **on**

One form, a few seconds. That bucket can hold petabytes, and costs nothing while empty.

On AWS: who may use it

AWS console → **IAM** → a user with an access key and **one** policy:

```
{"Version": "2012-10-17",
 "Statement": [{
   "Effect": "Allow",
   "Action": ["s3:GetObject", "s3:PutObject", "s3:ListBucket"],
   "Resource": ["arn:aws:s3:::ub-ads-taxi-2026",
                "arn:aws:s3:::ub-ads-taxi-2026/*"]}]}
```

Read, write and list **this bucket**. Nothing else: no deleting it, no other buckets, no machines. **Least privilege**.

On AWS: the same code

New terminal, real credentials, **no** `S3_ENDPOINT`:

```
export AWS_ACCESS_KEY_ID=AKIA...    AWS_SECRET_ACCESS_KEY=...
export AWS_REGION=eu-south-2        S3_BUCKET=ub-ads-taxi-2026
```

```
s3 = fs.S3FileSystem(endpoint_override=os.environ.get("S3_ENDPOINT"))
bucket = os.environ["S3_BUCKET"]

fs.copy_files("data/trips.parquet", f"{bucket}/raw/trips.parquet",
              destination_filesystem=s3)
pd.read_parquet(f"{bucket}/raw/trips.parquet", filesystem=s3)
```

Then the partitioning lines from Part 3, unchanged.

On AWS: a machine

AWS console → **EC2** → **Launch instance**

- **Image**: Ubuntu 24.04
- **Type**: `t3.micro`: 2 vCPU, 1 GB RAM, about a cent per hour
- **Key pair**: create one, download `ub-demo.pem`
- **Network**: allow SSH (22) and port 5001, **from your IP only**
- **IAM role**: one that may read the bucket, created beforehand in IAM

Launch. About a minute later: a Linux machine in a data centre, with a public IP.

On AWS: run the app on it

```
chmod 400 ub-demo.pem
scp -i ub-demo.pem -r app ubuntu@<public-ip>:
ssh -i ub-demo.pem ubuntu@<public-ip>
```

On the machine:

```
curl -fsSL https://get.docker.com | sudo sh
cd app && sudo docker build -t taxi-app .
sudo docker run -d -p 5001:5000 \
  -e AWS_REGION=eu-south-2 -e S3_BUCKET=ub-ads-taxi-2026 taxi-app
```

Open `http://<public-ip>:5001/trips/2024-01-15`.

On AWS: clean up

The part everyone forgets:

- **EC2** → Terminate the instance (a stopped instance still pays for its disk)
- **S3** → Empty and delete the bucket
- **IAM** → Delete the access key

All of the above: a few cents. Leaving the machine on for a year: around 100 USD. Hence the **budget alert**.

Azure from Python

pyarrow speaks Azure Blob Storage too:

```
az = fs.AzureFileSystem(
    account_name="<storage-account>",
    account_key=os.environ["AZURE_STORAGE_KEY"],
)

az.create_dir("taxi")
fs.copy_files("data/trips.parquet", "taxi/raw/trips.parquet",
              destination_filesystem=az)
pd.read_parquet("taxi/raw/trips.parquet", filesystem=az)
```

Different provider, different filesystem object. **The pandas line is the same.**

Storage classes and lifecycle

Not all data is read equally often:

Class (AWS names)	Storage price	Reading it
Standard	highest	instant, cheap
Infrequent Access	~half	instant, per-GB retrieval fee
Glacier	a fraction	minutes to hours, retrieval fee

Lifecycle rules move objects automatically: "after 30 days to Infrequent Access, after a year to Glacier, delete after 5 years".

GCS and Azure have the same idea with different names.

Compute services

	What you hand over	Good for
Virtual machines (EC2, Compute Engine)	Nothing, you get a Linux box	Full control, long jobs, GPUs
Containers (Fargate, Cloud Run, Container Apps)	An image	Apps and APIs: your session 2 image
Serverless functions (Lambda, Cloud Functions)	A function	Small, event-driven jobs
Managed ML (SageMaker, Vertex AI, Azure ML)	Training code	Training and hosting models

Move down the table: less to manage, less control.

Databases as a service

- **Relational**: RDS, Cloud SQL, Azure Database. The PostgreSQL from
  session 2, with backups and upgrades done for you
- **NoSQL**: DynamoDB, Firestore, Cosmos DB
- **Data warehouses**: BigQuery, Redshift, Snowflake. SQL over terabytes; most store data as columnar files on object storage

The warehouse pattern is today's class at scale: **columnar files in object storage, compute that reads only what it needs**.

Infrastructure as Code

Clicking around a console is not reproducible. Describe the infrastructure in a file, keep it in Git:

```
resource "aws_s3_bucket" "taxi" {
  bucket = "ub-agile-ds-taxi-2026"
}

resource "aws_instance" "worker" {
  ami           = "ami-0123456789abcdef0"
  instance_type = "t3.micro"
}
```

Query object storage with SQL

DuckDB reads Parquet on S3 directly, and uses partitions and column pruning on its own:

```
INSTALL httpfs; LOAD httpfs;
CREATE SECRET (TYPE s3, KEY_ID 'student', SECRET 'student-secret',
               ENDPOINT 'localhost:9000', URL_STYLE 'path', USE_SSL false);

SELECT pickup_day, count(*) AS trips
FROM read_parquet('s3://taxi/by_day/*/*.parquet', hive_partitioning = true)
GROUP BY pickup_day ORDER BY trips DESC LIMIT 3;
```

No server, no cluster: a single binary on your laptop.

Table formats: Iceberg and Delta

A folder of Parquet files has no transactions: two writers at once, or a half-finished write, and readers see a mess.

Table formats (Apache Iceberg, Delta Lake) add a metadata layer on top of the same Parquet files:

- Atomic commits, so readers never see half a write
- Schema evolution, time travel ("the table as of last Tuesday")
- Many engines on one copy of the data

This is the "lakehouse". Covered in depth in the Big Data course.

Where to learn more

- https://parquet.apache.org/docs/: the format
- https://arrow.apache.org/docs/python/filesystems.html: pyarrow and S3
- https://docs.rustfs.com/: RustFS
- https://docs.aws.amazon.com/AmazonS3/latest/userguide/: S3 concepts
- https://www.nyc.gov/site/tlc/about/tlc-trip-record-data.page: the taxi data, many more months

Questions?

See you on 21/10 with MLOps and GitHub Actions